

Arquitectura y Funcionamiento de Baileys (WhatsApp Web Protocol)

Documento de referencia técnica y arquitectura

Proyecto: **Neifert Automotores** — Módulo de WhatsApp CRM

Ubicación: [Whatsapp/Baileys.md](#)

Fecha: Septiembre 2026

1. Baileys no es una API REST: Es una Conexión WebSocket

Existe una confusión común al pensar en Baileys como una "API REST externa" (similar a Stripe, Twilio o MercadoPago) donde cada mensaje dispararía un request HTTP aislado. **No funciona así.**

WhatsApp no provee una API HTTP pública para cuentas estándar de WhatsApp Web. Baileys es una implementación completa en Node.js del **protocolo interno de WhatsApp Web (Companion Device)** mediante WebSockets y criptografía de extremo a extremo.



1.1. Cómo funciona la conexión bajo el capó

- **Una única conexión persistente:** Al iniciar (`whatsapp.js: iniciar()`), Baileys abre un único socket a `wss://web.whatsapp.com/ws/chat`. La conexión se

mantiene viva 24/7 mediante frames de *keep-alive* (ping/pong) cada 30 segundos.

- **Handshake Noise Protocol:** La autenticación y el canal seguro inicial se negocian utilizando el protocolo criptográfico *Noise* con las llaves guardadas en `data/session/` (`useMultiFileAuthState`).
- **Recepción de mensajes (Push en tiempo real):**
No hay polling ni peticiones periódicas. Cuando alguien envía un mensaje al número de WhatsApp, el servidor de WhatsApp empuja un frame binario directamente por el WebSocket abierto. Baileys lo descifra localmente y emite el evento interno `messages.upsert`.
- **Envío de mensajes (`sendMessage`):**
Para enviar un mensaje:
 1. Baileys cifra el contenido usando el **Signal Protocol** (Double Ratchet Algorithm).
 2. Debido a la arquitectura multi-dispositivo de WhatsApp, **cifra una copia independiente para cada uno de los dispositivos vinculados del destinatario** y para los propios dispositivos de la cuenta.
 3. Escribe un frame binario en el WebSocket y queda a la espera del **ACK** (acuse de recibo del servidor).

1.2. Por qué existe una latencia de ~1 segundo al enviar (y la solución)

El retardo de aproximadamente 1 segundo que se observa al despachar un mensaje **no es lentitud del servidor ni de la red local:**

- Es el tiempo matemático requerido para:
 1. Cifrar con Signal Protocol para N llaves públicas de dispositivos.
 2. El viaje de ida hacia los servidores de WhatsApp vía WebSocket.
 3. El procesamiento y confirmación (ACK) de WhatsApp.

Regla de UX — La Burbuja Optimista:

Este segundo no se puede reducir mediante optimizaciones de código porque depende de la criptografía y del protocolo de WhatsApp.

Por este motivo, la interfaz no debe esperar a que el servidor responda para mostrar el mensaje. La solución estándar es **mostrar la burbuja en la UI de inmediato en estado "pendiente"** (ícono de reloj) y actualizarla a "enviado" (un tilde gris) cuando el WebSocket confirme el ACK.

2. Supabase: Cuello de Botella Real vs. Límites Teóricos

A menudo se asume que los límites de mensajes por segundo de una base de datos o servicio en la nube (como Supabase) serán un problema para el chat. Los números reales demuestran lo contrario:

2.1. Límites de Supabase Realtime

Plan	Conexiones concurrentes	Mensajes por segundo	Channel joins por segundo
Free	200	100 / seg	100 / seg
Pro	500	500 / seg	500 / seg
Team	10.000	2.500 / seg	2.500 / seg

2.2. Métricas y volumen real de Neifert Automotores

- **Volumen anual analizado:** ~22.064 mensajes en el último año.
- **Promedio diario:** ~60 mensajes por día.
- **Tasa de tráfico real:** ~0,0007 mensajes por segundo.
- **Margen frente al plan Free:** El plan gratuito tolera 100 msgs/segundo; Neifert opera a 0,0007 msgs/segundo (**un margen de 140.000 veces**). Incluso con un crecimiento de 100x del negocio, el límite ni se roza.
- **Tamaño del historial textual:** 22.000 mensajes × ~450 bytes ≈ **10 MB anuales** (el plan Free de Supabase otorga 500 MB de base de datos relacional).

2.3. Dónde está el VERDADERO cuello de botella: La sincronización inicial

El riesgo real no está en la operación cotidiana, sino en el evento de **sincronización inicial del historial** ([messaging-history.set](#)):

- Al vincular la sesión por primera vez, WhatsApp empuja de golpe decenas de miles de mensajes históricos.
- **El error clásico:** Hacer un **INSERT** HTTP individual a la API PostgREST de Supabase por cada mensaje recibido:

$$22.000 \text{ round-trips} \times 50 \text{ ms} \approx 18 \text{ minutos de bloqueo}$$

Esto satura el pool de conexiones de Node.js y agota los límites de peticiones HTTP.

- **La solución correcta: Inserción en lotes (Batch Inserts).**
Agrupar los mensajes históricos en bloques de 500 filas por request reduce las 22.000 peticiones a solo ~44 requests HTTP, completando la sincronización en cuestión de segundos.

2.4. Manejo de multimedia (Fotos, Videos, Audios, Documentos)

Los archivos binarios **NUNCA** deben guardarse dentro de la base de datos relacional (ni como base64 ni como bytea):

- Se almacenan en buckets de objetos: **Cloudflare R2** (sin costos de egreso) o **Supabase Storage**.
- La base de datos relacional únicamente conserva la metadata:

```
{
  "id": "3EB0...",
  "chat_id": "5493564...@s.whatsapp.net",
  "tipo": "imagen",
  "url": "https://r2.neifert.com.ar/media/3EB0....jpg",
  "tamano": 1048576,
  "mime": "image/jpeg"
}
```

3. Varias PC con el Mismo Número: Arquitectura y Restricciones

3.1. Restricciones duras del protocolo WhatsApp

1. Límite de 4 dispositivos vinculados:

WhatsApp solo permite vincular un máximo de 4 dispositivos acompañantes (*companion devices*) por número telefónico. Cada instancia independiente de Baileys consume 1 slot completo de esos 4.

2. Conflicto de sesión (Error 440 `connectionReplaced`):

Si dos procesos de Node intentan conectarse usando la misma carpeta de credenciales (`data/sesion/`), los servidores de WhatsApp detectan la colisión y desconectan de inmediato a una de las instancias con el error 440 `connectionReplaced`.

3. La regla de los 14 días:

El teléfono celular principal **debe abrir WhatsApp al menos una vez cada 14 días con conexión a internet**. Si transcurren 14 días sin actividad en el teléfono móvil, WhatsApp desvincula automáticamente por seguridad a todos los dispositivos vinculados. Esto es crítico para servidores desatendidos 24/7.

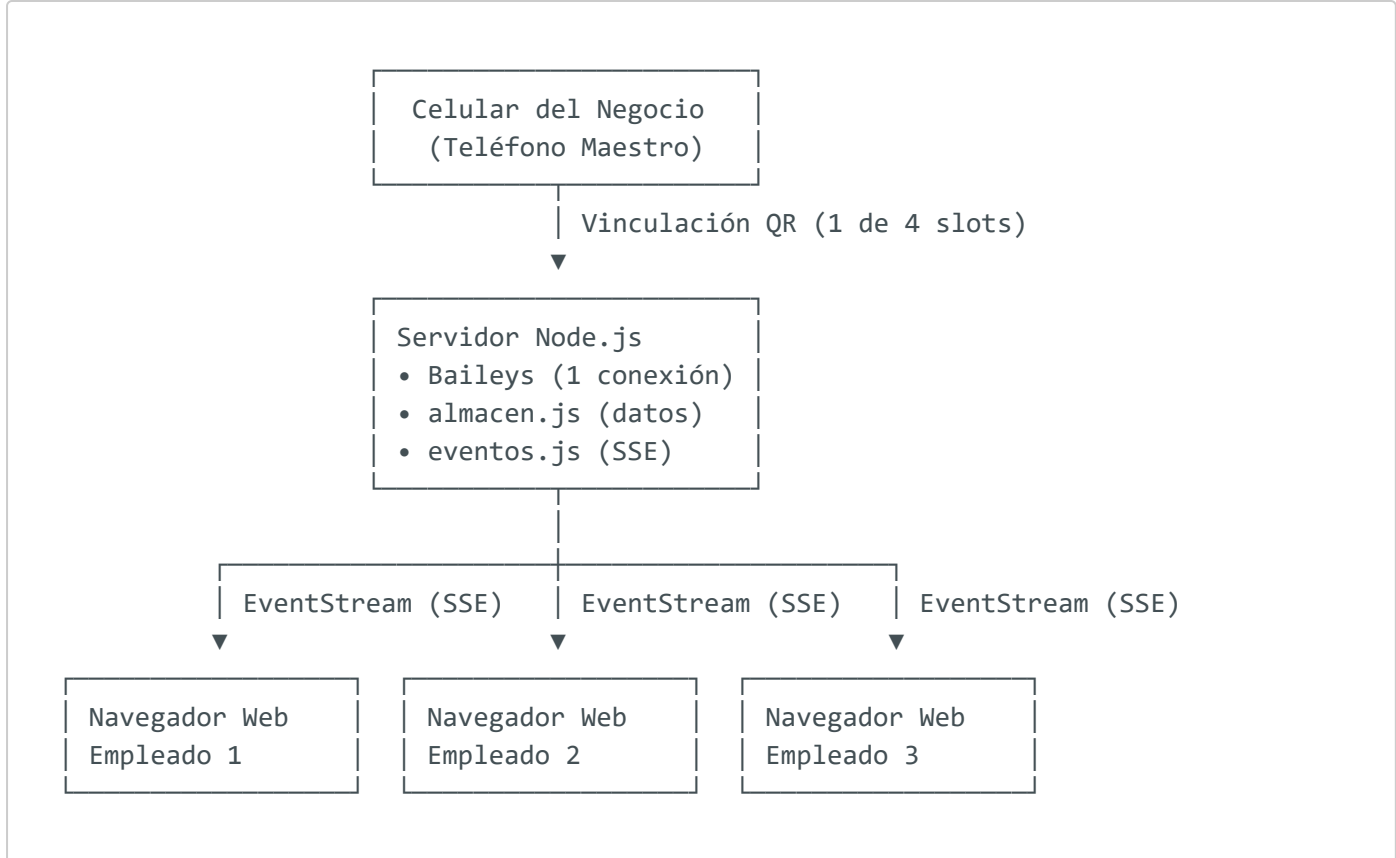
3.2. Por qué no se debe instalar Baileys en cada PC

Instalar Baileys de forma independiente en 3 computadoras de los empleados causaría los siguientes problemas:

- Se agotarían 3 de los 4 slots de vinculación disponibles en la cuenta.
- Cada PC tendría su propio historial local desincronizado.
- Si el empleado A lee un mensaje, en las computadoras de B y C seguiría apareciendo como no leído.
- Se multiplicarían por 3 los riesgos de baneo y consumo de recursos.

3.3. Arquitectura correcta: Un solo Baileys + Clientes vía SSE

La arquitectura adecuada consiste en **un único servicio central de Node.js con Baileys** y N navegadores de empleados conectados al panel:



Ventajas ya operativas en el servidor actual:

- El archivo `eventos.js` implementa un canal de **Server-Sent Events (SSE)** con `emitir(evento, datos)` transmitiendo en broadcast a todos los navegadores conectados.
- Cuando el Empleado 1 envía un mensaje o ingresa un mensaje de un cliente, el servidor lo procesa una sola vez y lo despacha instantáneamente por SSE a todos los empleados conectados.

4. Requisitos para el Escenario Multi-Operador en Producción

Para que tres empleados puedan trabajar en simultáneo sin conflictos sobre la misma línea de WhatsApp, se deben implementar cuatro puntos:

4.1. Abrir el puerto y agregar Autenticación

- Actualmente en `src/config.js`, el servicio está configurado como `HOST = '127.0.0.1'` para evitar accesos indebidos durante el desarrollo local.

- Para operar en red local (LAN) o en un VPS externo, se debe cambiar **HOST** a **0.0.0.0** y **agregar obligatoriamente autenticación** (usuarios, contraseñas o tokens JWT). Sin autenticación, cualquier dispositivo con acceso a la red podría enviar mensajes haciéndose pasar por la concesionaria.

4.2. Atribución de autoría interna (Quién respondió)

Actualmente todos los mensajes enviados desde el panel figuran con autor **"yo"**. En un entorno multi-usuario, el esquema de datos debe registrar qué usuario interno redactó el mensaje:

```
{
  "id": "3EB0ABC123...",
  "deMi": true,
  "autorInterno": "Julián (Ventas)",
  "usuarioId": "usr_9481",
  "texto": "Hola, sí, la Hilux sigue disponible.",
  "ts": 1774372587
}
```

4.3. Prevención de colisiones ("Escribiendo...")

Para evitar que dos vendedores contesten al mismo tiempo la misma consulta a un cliente:

- Aprovechar el canal SSE existente para propagar un evento **operador_escribiendo**:

```
{ "chatId": "5493564...@s.whatsapp.net", "operador": "Marcos", "estado":
  "componiendo" }
```

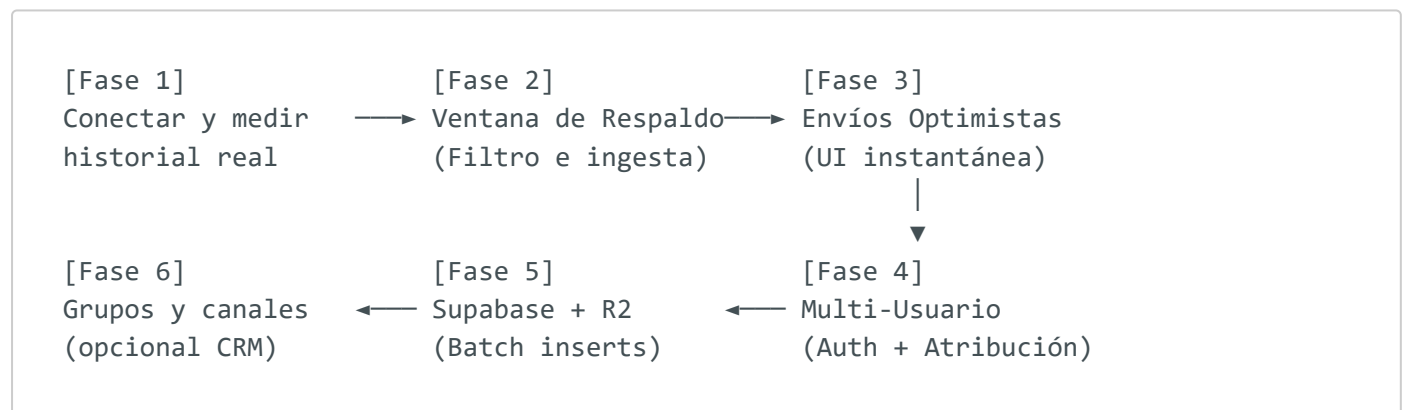
- La pantalla de los demás empleados muestra: *"Marcos está respondiendo este chat..."*, bloqueando temporalmente el envío o alertando al segundo operador.

4.4. Riesgo de Baileys vs. WhatsApp Cloud API Oficial

- **Baileys (No oficial):** Es gratuito y no tiene costos por mensaje ni limitaciones de plantillas, pero está sujeto a cambios en el protocolo de WhatsApp, requiere mantener la regla de los 14 días en el teléfono y existe riesgo de baneo si se realizan envíos masivos o conductas que violen las políticas de Meta.
- **WhatsApp Cloud API (Oficial):** Permite multi-agente nativo sin límite de terminales ni teléfono celular encendido, pero tiene costo por ventana de conversación de 24 horas y exige validación de plantillas (templates) para iniciar conversaciones salientes.

5. Hoja de Ruta de Implementación Técnica (Roadmap)

El orden de trabajo recomendado para evitar retrabajos es el siguiente:



1. Fase 1: Vinculación e ingesta del historial base:

Vincular el número real o de prueba y dejar que descargue el historial para medir el volumen exacto de mensajes y multimedia existente.

2. Fase 2: Ventana de respaldo (Filtro y purga periódica):

Establecer la política de retención: qué chats se procesan, descartar contenido innecesario (estados, llamadas, newsletters) y configurar limpieza programada de archivos temporales.

3. Fase 3: Envío con burbuja optimista en frontend:

Pintar el mensaje inmediatamente en la interfaz al pulsar Enter y actualizar su estado conforme lleguen los eventos del socket.

4. **Fase 4: Multi-usuario y atribución antes de la base de datos:**
Definir roles de empleados, login y campo de autor en los mensajes. Esto debe hacerse antes de migrar a base de datos para no tener que hacer migraciones de esquemas posteriores.
5. **Fase 5: Conexión con Supabase y Cloudflare R2:**
Reemplazar la persistencia local de `src/almacen.js` por la capa de base de datos con inserciones por lote (*batching*) y subir las fotos/audios a buckets R2.
6. **Fase 6: Canales y grupos secundarios:**
Evaluar si se incorporan chats grupales al CRM (actualmente filtrados para no saturar con mensajes irrelevantes de grupos).

6. Despliegue: ¿PC Local o Servidor VPS?

La elección del entorno de ejecución define la necesidad inmediata de Supabase:

Criterio	Opción A: PC fija en la Concesionaria	Opción B: Servidor VPS en la Nube (Ubuntu/Docker)
Acceso	Solo dentro de la oficina (red local WiFi)	Desde cualquier lugar (oficina, home office, celular)
Persistencia	Los archivos JSONL locales (<code>data/</code>) alcanzan y son ultra veloces	Requiere base de datos centralizada (Supabase) y Storage (R2)
Disponibilidad	Si se apaga la PC o se corta la luz, se corta WhatsApp	99.9% uptime 24/7 en centro de datos
Costo	\$0 adicional	~5a10 USD/mes de servidor
Regla de 14 días	Fácil de monitorear (el teléfono suele estar cerca)	Se debe recordar al encargado abrir la app cada 10-12 días

7. Mapeo de Componentes del Servidor (`whatsapp/servidor/`)

Archivo	Responsabilidad
<code>src/config.js</code>	Variables de entorno, puertos, rutas de datos y límites de multimedia.
<code>src/whatsapp.js</code>	Conexión con Baileys (<code>makeWASocket</code>), gestión del ciclo de vida del socket, eventos de mensajes, reconexión exponencial y cifrado.
<code>src/almacen.js</code>	Capa de persistencia única (actualmente archivos JSON y JSONL). Este es el único módulo que se sustituye para integrar Supabase.
<code>src/eventos.js</code>	Servidor de Server-Sent Events (SSE) y registro de actividad en tiempo real hacia los navegadores.
<code>src/fotos.js</code>	Descarga diferida y almacenamiento en caché de avatares de perfil.
<code>src/audio.js</code>	Transcodificación de notas de voz a formato OGG Opus (PTT - Push to Talk).
<code>index.js</code>	API HTTP Express y orquestación de arranque y apagado seguro (<i>graceful shutdown</i>).